CMSC 35200 — Lab 1: Two Coding Agents, One Model, Measured Evidence

Deep Learning Systems (DLS-2026)

Due: Thursday, October 8, 2026, 11:59 PM (Chicago)

Contents

CMSC 35200 — Lab 1: Two Coding Agents, One Model, Measured Evidence **1**
- Learning objectives . . . 2
- Time, equipment, and due date . . . 2
- Course service contract . . . 2
- Part A — Install and configure both agents (20 points) . . . 3
  - A1. Prepare your workspace . . . 3
  - A2. Install Pi . . . 3
  - A3. Install Codex CLI . . . 3
  - A4. Authority settings (required for both agents) . . . 4
- Part B — Three paired agent tasks (24 points) . . . 5
  - Task 1 — Evidence-backed CLI repair . . . 5
  - Task 2 — Small data-analysis pipeline . . . 5
  - Task 3 — LaTeX report toolchain repair . . . 5
- Part C — pi-30 paired benchmark experiment (36 points) . . . 5
  - C1. Obtain the course package . . . 5
  - C2. Experimental design . . . 6
  - C3. Required analyses . . . 6
- Part D — Agent-authored technical report (20 points) . . . 7
- Submission checklist . . . 7
- Release gate for course staff (not student work) . . . 7

Lab 1 Rubric — 100 points **9**
- Scoring principles . . . 10

CMSC 35200 — Lab 1: Two Coding Agents, One Model, Measured Evidence

Release status: Instructor draft — do **not** release until the endpoint and both agent adapters pass the release-gate checklist at the end of this document.

Individual assignment. Discuss setup problems with classmates, but do not share agent transcripts, source edits, benchmark outputs, or report prose. Submit your own artifacts and account of every agent action.

Learning objectives

By the end of this lab you will be able to:

1. install and operate two terminal coding agents, **Pi** and **Codex CLI**;
2. connect both to the course's OpenAI-compatible Qwen3.8-27B service;
3. specify bounded agent authority, inspect the work an agent performs, and verify results independently;
4. conduct a controlled agent comparison using verifier-backed coding tasks; and
5. produce a concise reproducible technical report in LaTeX and PDF.

This is an experiment about agent systems, **not** a contest to obtain the highest score. Negative results, failures, and incompatibilities receive full credit when measured and explained honestly.

Time, equipment, and due date

- Expected effort: **8–12 hours**.
- Work on your own laptop/desktop or a personal remote machine. macOS, Linux, and Windows with WSL2 are supported.
- Local GPU hardware is not required.
- Due date: **Thursday, October 8, 2026, 11:59 PM (Chicago time)**. Canvas submission link: **posted on the CMSC 35200 Canvas site under Assignments → "Lab 1: Two Coding Agents"** (open the course Canvas page and use that assignment's file-upload submission).
- Use only the allocated course endpoint and quota. Do not substitute a paid model or a personal API key for the comparison run.

Course service contract

Field	Course value
Service	Chicago-6 DLS LiteLLM aggregator
Base URL	`http://209.182.219.81:4010/v1`
Model	`qwen3.8-27b`
API style	OpenAI-compatible Chat Completions
Access	Obtain access instructions and the course key from Canvas
Context window	262,144 tokens
Special behavior	Reasoning may be returned separately from final content

Never commit or paste the course key into a repository, report, transcript, screenshot, or public chat. Store it only in a shell environment variable or approved credential store:

```
export DLS_QWEN_API_KEY='value supplied privately in Canvas'
export DLS_QWEN_BASE_URL='http://209.182.219.81:4010/v1'
export DLS_QWEN_MODEL='qwen3.8-27b'
```

Before configuring either agent, make a smoke-test request. Save the *redacted* response headers/body and exact command in `evidence/endpoint-smoke.txt`.

```
curl -sS --fail-with-body "$DLS_QWEN_BASE_URL/models" \
  -H "Authorization: Bearer $DLS_QWEN_API_KEY" > evidence/models.json
curl -sS --fail-with-body "$DLS_QWEN_BASE_URL/chat/completions" \
  -H "Authorization: Bearer $DLS_QWEN_API_KEY" \
  -H 'Content-Type: application/json' \
  -d "{\"model\":\"$DLS_QWEN_MODEL\",\"messages\":[{\"role\":\"user\",\"content\":\"Reply with
  > evidence/chat-smoke.json
```

A `/v1/models` success does **not** prove the service can generate. The chat-completion request must return final content. If it fails, record the status/error and contact course staff; do not quietly change endpoint or model.

Part A — Install and configure both agents (20 points)

A1. Prepare your workspace

Install Git, a current Node.js LTS release, Python 3, and a LaTeX distribution (`latexmk` plus `pdflatex` or equivalent). Create a private working directory:

```
mkdir -p ~/cmsc35200/lab1/{evidence,runs,report}
cd ~/cmsc35200/lab1
git init
```

Record OS, CPU, RAM, shell, Git, Node, Python, Pi, Codex, and LaTeX versions in `evidence/environment.txt`.

A2. Install Pi

Install the current Pi Coding Agent package, then record its version:

```
npm install --global @earendil-works/pi-coding-agent
pi --version
```

> **PATH note.** `npm install --global` places `pi` (and later `codex`) in your npm global bin directory (`npm prefix -g`/bin, e.g. `~/.npm-global/bin` or `/usr/local/bin`). If `pi: command not found`, add that directory to your `PATH` (e.g. `export PATH="$(npm prefix -g)/bin:$PATH"` in your shell profile) and re-open the shell.

Use `scripts/configure_pi_provider.py` from the course starter package to add a provider named `dls-qwen` using environment variables—not a hard-coded key. Verify a non-interactive smoke test:

```
pi --provider dls-qwen --model qwen3.8-27b --print \
  'Reply with exactly PI_CONNECTED'
```

Pi must be able to use the local tools needed for this lab: file read/write/edit, shell commands, and an approved web-search capability. Your transcript must identify the search mechanism you enabled (for example, the approved course extension/MCP server) and show one search that helped solve a task.

A3. Install Codex CLI

Install Codex CLI and record its version:

```
npm install --global @openai/codex
codex --version
```

Copy `templates/codex-dls-qwen.config.toml` to the **user-level** Codex configuration location (`~/.codex/config.toml`), then set `DLS_QWEN_API_KEY` in your shell. Do not put custom-provider configuration only in a project-local `.codex/config.toml`: current Codex deliberately ignores provider/auth routing there.

> **Codex protocol requirement (important).** Current Codex (0.158+/0.160) **no longer supports `wire_api = "chat"`** — a config using it will not load (see openai/codex discussion #7782). The course provider therefore uses **`wire_api = "responses"`**, and the course endpoint (Chicago-6 LiteLLM) serves a compatible OpenAI **Responses** bridge at `/v1/responses`. The supplied template is already set to `wire_api = "responses"`; do not change it to `chat`.

Verify the configuration with a bounded run from your lab directory:

```
cd ~/cmsc35200/lab1
codex exec --sandbox read-only \
  --output-last-message evidence/codex-smoke.txt \
  'Reply with exactly CODEX_CONNECTED. Do not modify files.'
```

For task work (Parts B/C), run Codex with web search and workspace-write under an approval policy, e.g.:

```
codex exec --search --sandbox workspace-write \
  -c approval_policy=on-request \
  --output-last-message evidence/codex-run.txt '<task prompt>'
```

> **Flag note.** Codex CLI flag names vary by version. On the installed version, approval is set via config (`-c approval_policy=on-request` or `--approve-for-me`), and `--approve-for-me` cannot be combined with `--sandbox`. Run `codex exec --help` to confirm the flags your version accepts. Codex must have web search enabled and permission to request installation of dependencies; it must not receive unrestricted machine authority (no `--dangerously-bypass-approvals-and-sandbox`).

A4. Authority settings (required for both agents)

For the three tasks and benchmark runs, permit agents to:

- read and edit only the `~/cmsc35200/lab1` workspace;
- execute ordinary development commands, tests, and project-local dependency installation;
- use the approved web-search capability for documentation and package research; and
- ask before commands that require privilege escalation, modify global configuration, delete outside the workspace, access credentials, send network data other than ordinary package/documentation/model requests, or publish anything.

You—not the agent—are responsible for reviewing the diff and verifier output. Do **not** use `sudo`, `--dangerously-bypass-approvals-and-sandbox`, or an equivalent unrestricted mode.

Part B — Three paired agent tasks (24 points)

Run **each task once with Pi and once with Codex** using the same task statement and same course model. Keep a separate clean directory for every agent/task pair. The agent may search the web and install project-local dependencies if needed. Save the prompts, command transcript/event log, final diff, and independent test output.

Task 1 — Evidence-backed CLI repair

Create a broken Python command-line program that reads a CSV of numeric values and prints count, mean, median, and sample standard deviation. Ask each agent to diagnose and repair it. Require unit tests for malformed rows, even/odd cardinality, and `n < 2`. You must write an independent test that the agent did not see before the run.

Task 2 — Small data-analysis pipeline

Give each agent the same small public CSV (cite its source). Ask it to build a reproducible Python analysis that validates the input, produces one figure and one summary table, and writes `README.md` instructions. Each agent must use web search to locate and cite one relevant package/documentation source. Independently rerun the pipeline in a clean directory.

Task 3 — LaTeX report toolchain repair

Give each agent a deliberately incomplete one-page LaTeX document containing a table, a figure reference, a citation, and at least one compilation error. Ask it to install only necessary project-local tooling, repair the document, and produce a PDF. Independently compile the resulting `.tex` with `latexmk -pdf`.

For all three tasks, document what the agent did well, what it did poorly, what you had to correct, and whether the final output passed your independent check.

Part C — pi-30 paired benchmark experiment (36 points)

> **Adapters are published.** The official Pi and Codex adapters live in the `adapters/` directory of the course `pi-30` repo (`rick-stevens-ai/pi-30`, commit `4e9148c` or later): `pi_adapter.py`, `codex_adapter.py`, the shared runner `common.py`, the canonical `runs_schema.py`, the 30-task spec `tasks_spec.json`, and the orchestrator `run_lab1_benchmark.sh`. Both were piloted end-to-end on Linux and emit the `runs.csv` schema below. See `adapters/README.md`.

C1. Obtain the course package

Clone the instructor-published, versioned package—not an arbitrary fork:

```
git clone https://github.com/rick-stevens-ai/pi-30.git pi-30
cd pi-30
git rev-parse HEAD          # record the commit you cloned
bash harness/assert_seeds_fail.sh
```

`harness/assert_seeds_fail.sh` must pass **before** any agent is allowed to work. It proves that the seed scaffolds are actually broken. Do not use solution files, prior run directories, hidden graders, or another student's artifacts as agent input.

C2. Experimental design

The course harness will run all 30 tasks using the same:

- pi-30 commit;
- Qwen model ID and endpoint;
- task prompt files and verifier/scorer code;
- maximum output-token budget, timeout, retry rule, and allowed tools;
- one fresh workspace per task and agent; and
- single-run order or a recorded randomized order.

Run the official **Pi adapter** and **Codex adapter** (in `adapters/`). The simplest path is the orchestrator, which runs the seed guard first, then both agents, then prints a summary:

```
# all 30 problems, both agents, into runs.csv (verifier exit code is the authority)
bash adapters/run_lab1_benchmark.sh

# or a slice while you validate your setup
bash adapters/run_lab1_benchmark.sh P1,P3,P10

# or one agent at a time
python3 adapters/pi_adapter.py    --tasks P1,P2 --csv ./runs.csv
python3 adapters/codex_adapter.py --tasks P1,P2 --csv ./runs.csv
```

The adapters write one row per attempt to `runs.csv`; do not score agent prose. The verifier/scorer exit code is the authority. Each adapter stages a fresh workspace per task, runs the task's iterate-until-green loop (feeding the verifier's failure output back on retries), and records telemetry. The Pi `--print` CLI does not expose token counts, so those fields are recorded as `unknown`; Codex reports a total-token figure, which is captured.

Required fields per attempt:

```
agent, task_id, attempt, start_utc, end_utc, wall_seconds,
status, verifier_status, score, prompt_tokens, completion_tokens,
reasoning_tokens, total_tokens, tool_calls, retry_count,
model, endpoint_label, pi_or_codex_version, harness_commit,
stdout_path, stderr_path, transcript_path
```

Use `unknown`, not an invented number, when a runtime does not expose a token field. Report availability separately for infrastructure failures, invalid output, task failures, verified successes, and not-run tasks.

C3. Required analyses

Your report must compare Pi and Codex using:

- completion count and rate, plus a 95% Wilson interval;
- verifier/scorer results by task and task family;
- median and total wall-clock time, clearly separating service wait from local work if available;
- total, median-per-task, and per-verified-success token use, with reasoning tokens reported separately when available;
- retries, failures, and tool calls; and

- at least three concrete transcript-level observations about planning, tool use, debugging, and failure recovery.

Do not claim that one agent is "better" from a single aggregate. State the controlled variables, confounders, missing telemetry, and limitations.

Part D — Agent-authored technical report (20 points)

Ask **both agents** to help draft and revise the report, but you remain the author responsible for its claims. Submit:

- `report/lab1_report.tex`;
- `report/lab1_report.pdf` (4–6 pages excluding appendix);
- bibliography/source file(s);
- the generated tables/figures and raw `runs.csv`;
- `report/build.log`; and
- a short `AGENT_DISCLOSURE.md` identifying which agent produced or revised each part.

Compile with:

```
cd report
latexmk -pdf -interaction=nonstopmode -halt-on-error lab1_report.tex
```

The report needs: abstract; setup; agent-authority policy; methods; results; failure analysis; limitations; conclusion; and a reproducibility appendix containing commands, versions, redacted configuration, commit IDs, and an artifact manifest.

Submission checklist

Submit one compressed directory or repository tag containing:

1. `evidence/` (environment, endpoint smoke test, redacted configurations, transcripts/logs);
2. task directories with prompts, source, diffs, and independent checks;
3. pi-30 `runs.csv`, raw adapter logs, and exact package/harness commit IDs;
4. `report/lab1_report.tex`, sources, build log, and PDF; and
5. `AGENT_DISCLOSURE.md` and `README.md` with replay instructions.

Release gate for course staff (not student work)

Do not publish this assignment until staff have recorded all of the following in the instructor pilot log:

- Chicago-6 `/v1/models` **and** `/v1/chat/completions` return a real completion;
- all eight intended backend instances are reachable or an explicit degraded-service policy is in place;
- Pi completes a live tool-use smoke run against the endpoint;
- Codex completes a live tool-use smoke run against the endpoint;
- the selected Codex provider protocol is supported by the installed Codex version (the endpoint is Chat Completions compatible; Codex Responses-only configurations will not work without an adapter);

- the released pi-30 repository has clean seeds, pinned prompts, public verifier boundaries, private grading boundaries, and official Pi/Codex adapters that emit the required telemetry; and
- a macOS, Linux, and Windows/WSL pilot has completed the entire lab within the stated budget.

Lab 1 Rubric — 100 points

Criterion	Points	Full-credit evidence	Partial credit	No credit
Pi installation and live configuration	10	Version recorded; redacted config; live final-content smoke test; tool/web-search capability demonstrated	Installed but configuration or evidence incomplete	No usable Pi evidence
Codex installation and live configuration	10	Version recorded; user-level custom provider/profile; live final-content smoke test; `--search` and approval policy shown	Installed but adapter/configuration incomplete	No usable Codex evidence
Security and authority policy	6	Clear bounded permissions; secrets redacted; no unrestricted mode; agent actions reviewed	Sensible policy but missing evidence or one avoidable exposure	Key exposed, unsafe permissions, or unreviewed destructive action
Three paired tasks: design and execution	12	Same task statements; separate workspaces; both agents used; prompts/transcripts/diffs preserved	Minor asymmetry or missing artifact	Tasks not paired or claimed without evidence
Three paired tasks: independent verification and analysis	12	Independent tests/builds; honest outcomes; concrete comparison of behavior and web/tool use	Some checks but weak comparison	Agent self-report substituted for verification

Criterion	Points	Full-credit evidence	Partial credit	No credit
pi-30 experimental control	10	Exact commits; seed guard passed first; equal budgets/settings; clean per-task workspaces; no leakage	Small reproducibility/control gap	Contaminated seeds, answer leakage, or incomparable conditions
pi-30 measurements and raw evidence	10	Complete `runs.csv`; logs/transcripts; wall time, tokens where available, retry/tool data, and explicit `unknown` fields	Some telemetry absent but transparently documented	Aggregate claims without raw evidence
pi-30 scoring and statistical analysis	10	Verifier-based status; completion/score results; intervals; time/token summaries; failures classified	Correct basic summaries but missing interval/failure accounting	Scores based on prose or altered tests
LaTeX report and PDF	10	4–6 pages; compiles cleanly; clear methods/results/limitations; figures/tables readable; bibliography present	Minor build/organization defect	No source or no rendered PDF
Reproducibility, disclosure, and judgment	10	Replay README, manifest, redacted settings, agent disclosure, limitations, and no unsupported conclusions	Good package with gaps	Incomplete or misleading provenance

Scoring principles

- **Evidence beats narrative.** A passing verifier, independent build, exact log, or raw record outranks any agent's claim.

- **Negative results can earn full credit.** A documented endpoint outage, failed adapter, or poor benchmark result is valuable if the student preserved evidence and did not conceal it.
- **No credit for contamination.** Copying a solution, modifying a verifier, using a hidden answer key as context, or reusing another student's outputs invalidates the affected portion.
- **Token accounting is descriptive, not a penalty.** Students are graded on accurate measurement and interpretation, not on spending the fewest tokens.
- **Accessibility/fallback.** A student blocked by endpoint availability must submit the local setup and failure evidence promptly; staff will provide an equivalent controlled make-up run.